

Python ile Veri Bilimi

Teoriden Uygulamaya Kapsamlı Rehber

NumPy • Pandas • Matplotlib • scikit-learn

Bu belge, veri bilimini Python ekosistemiyle — **teorisi ve çalışan kod örnekleriyle** — baştan sona anlatır. Sayısal hesaplamadan (NumPy) veri işlemeye (Pandas), görselleştirmeden (Matplotlib/Seaborn) istatistik ve makine öğrenmesine (scikit-learn) kadar tam bir veri bilimi hattını kapsar.

Kapsam — Kısım 1 (Temeller & NumPy): veri bilimi iş akışı • Python ekosistemi • ndarray • dtype • yayınlama (broadcasting) • vektörleştirme • indeksleme • doğrusal cebir & rastgelelik.

Kapsam — Kısım 2 (Pandas): Series & DataFrame • veri okuma/yazma • seçme & filtreleme • veri temizleme • groupby & pivot • birleştirme • zaman serileri.

Kapsam — Kısım 3 (Görselleştirme): Matplotlib temelleri • alt grafikler & stil • Seaborn ile istatistiksel grafikler.

Kapsam — Kısım 4 (İstatistik): betimsel istatistik • olasılık dağılımları • korelasyon • hipotez testleri.

Kapsam — Kısım 5 (Makine Öğrenmesi): veri hazırlama • regresyon • sınıflandırma • kümeleme • model değerlendirme • pipeline'lar.

Hazırlanma Tarihi: 12 Temmuz 2026

Python 3 • NumPy • Pandas • scikit-learn

Contents

1	Veri Bilimine Giriş	4
1.1	Veri Bilimi İş Akışı	4
1.2	Python Veri Bilimi Ekosistemi	4
1.3	Kurulum ve Ortam	4
2	NumPy: Sayısal Hesaplamanın Temeli	5
2.1	Neden NumPy? Liste vs Dizi	5
2.2	Dizi Oluşturma	5
2.3	Dizi Özellikleri: shape, dtype, boyut	6
2.4	Yeniden Şekillendirme (Reshape)	6
3	NumPy: İndeksleme, Yayınlama ve İşlemler	6
3.1	İndeksleme ve Dilimleme	6
3.2	Yayınlama (Broadcasting)	7
3.3	Eksen (axis) Boyunca İşlemler	7
3.4	Doğrusal Cebir	7
3.5	Rastgelelik ve Tekrarlanabilirlik	8
4	NumPy Derinlemesine: Veri Tipleri ve Bellek	8
4.1	Veri Tipleri (dtype) Derinlemesine	8
4.2	Görünüm (View) mü, Kopya (Copy) mı?	9
4.3	Bir Diziyi Yerinde Değiştirmek	9
5	NumPy Derinlemesine: Vektörizasyon ve Performans	10
5.1	Evrensel Fonksiyonlar (ufunc) ve Vektörizasyon	10
5.2	Dizi Birleştirme ve Bölme	11
5.3	Sıralama, Arama ve Faydalı İşlemler	11
5.4	Neden Bu Kadar Hızlı? Bellek Düzeni	11
6	Pandas Temelleri: Series ve DataFrame	13
6.1	Series: Etiketli Tek Boyutlu Veri	13
6.2	DataFrame: Tablo Verisi	13
6.3	İlk Bakış: describe ve info	14
7	Veri Okuma ve Yazma	14
8	Veri Seçme ve Filtreleme	14
8.1	Sütun ve Satır Seçme	15
8.2	Boole Filtreleme (En Önemli Beceri)	15
8.3	Yeni Sütun Oluşturma ve Dönüştürme	15
9	Veri Temizleme	16
9.1	Eksik Veri (NaN) ile Başa Çıkma	16
9.2	Tekrarlar ve Tip Dönüşümleri	16
10	Gruplama ve Toplulaştırma (GroupBy)	17
10.1	Pivot Tablolar	17
11	Veri Birleştirme	17
11.1	merge: Anahtar Üzerinden Birleştirme	17
11.2	concat: Uç Uca Ekleme	18

12 Zaman Serileri	18
13 Matplotlib: Temel Çizim	20
13.1 Anatomi: Figure ve Axes	20
13.2 Temel Grafik Türleri	20
13.3 Birden Çok Alt Grafik (Subplots)	21
13.4 Stil ve Özelleştirme	21
14 Matplotlib Derinlemesine: Grafik Türleri Kataloğu	22
14.1 Çizgi Grafik: Birden Çok Seri ve Stil	22
14.2 Dağılım Grafiği: Renk ve Boyutla Kodlama	22
14.3 Çubuk Grafik: Gruplu ve Yığılmış	22
14.4 Histogram ve Yoğunluk: Dağılımı Görmek	23
14.5 Kutu ve Keman: Grupları Karşılaştırmak	23
15 Matplotlib Derinlemesine: Eksen, Yazı ve Kaydetme	23
15.1 Eksenleri Tam Kontrol: Sınırlar, İşaretler, Ölçek	23
15.2 Metin, Ok ve Vurgulama (Annotation)	24
15.3 İkiz Eksen (Twin Axes): Farklı Birimler	24
15.4 Karmaşık Yerleşim ve Yüksek Kaliteli Kaydetme	24
16 Pandas ile Hızlı Çizim	25
17 Seaborn: İstatistiksel Görselleştirme	25
17.1 Dağılım Grafikleri	25
17.2 İlişki Grafikleri	26
17.3 Kategorik Grafikler	26
18 Betimsel İstatistik	27
18.1 Merkezi Eğilim: Ortalama, Medyan, Mod	27
18.2 Yayılm: Varyans, Standart Sapma, Çeyrekler	27
19 Olasılık Dağılımları	28
19.1 Normal (Gauss) Dağılımı	28
19.2 Diğer Önemli Dağılımlar	28
19.3 Merkezi Limit Teoremi	29
20 Korelasyon ve Hipotez Testleri	29
20.1 Korelasyon: İlişki Ölçümü	29
20.2 Hipotez Testleri	29
21 Makine Öğrenmesine Giriş	31
21.1 ML Türleri	31
21.2 scikit-learn'in Tutarlı Arayüzü	31
21.3 Eğitim/Test Ayrımı: Altın Kural	31
22 Regresyon	32
22.1 Doğrusal Regresyon	32
22.2 Regresyon Metrikleri	32
22.3 Aşırı Öğrenme ve Düzenleme (Regularization)	33
23 Sınıflandırma	33
23.1 Lojistik Regresyon ve Diğer Modeller	33
23.2 Sınıflandırma Metrikleri ve Karmaşıklık Matrisi	33

24 Denetimsiz Öğrenme: Kümeleme ve PCA	34
24.1 K-Means Kümeleme	34
24.2 PCA: Boyut İndirgeme	34
25 Model Değerlendirme ve Pipeline’lar	35
25.1 Çapraz Doğrulama (Cross-Validation)	35
25.2 Hiperparametre Ayarı: GridSearch	35
25.3 Pipeline: Ön İşleme + Model Tek Zincirde	35
26 Proje 1 — Palmer Penguins: Keşifsel Veri Analizi	37
27 Proje 2 — Titanic: Temizleme ve Sınıflandırma	38
28 Proje 3 — Tips: İstatistik ve Görselleştirme	40
29 Proje 4 — California Housing: Regresyon	41
30 Kapanış: Uçtan Uca Bir Proje	42
30.1 Özet: Doğru Aracı Seçmek	43

KISIM 1

Temeller ve NumPy

1. Veri Bilimine Giriş

Veri bilimi, ham veriden anlamlı bilgi ve öngörü çıkarma disiplindir; istatistik, programlama ve alan bilgisinin keşişiminde durur. Bir veri bilimi projesi nadiren “model eğitmek”ten ibarettir; zamanın büyük kısmı veriyi toplamak, temizlemek ve anlamakla geçer.

1.1 Veri Bilimi İş Akışı

Tipik bir proje şu aşamalardan geçer. Bu rehber, her aşama için gereken araçları sırayla öğretir.

Aşama	Ne yapılır?	Araç
1. Toplama	Veriyi kaynaklardan al (CSV, SQL, API)	Pandas
2. Temizleme	Eksik/hatalı/tekrar veriyi düzelt	Pandas
3. Keşif (EDA)	İstatistik ve grafiklerle veriyi anla	Pandas, Matplotlib
4. Modelleme	İstatistik/ML modeli kur	scikit-learn
5. Değerlendirme	Modeli ölç, yorumla	scikit-learn
6. İletişim	Sonuçları görselleştir, sun	Matplotlib, Seaborn

1.2 Python Veri Bilimi Ekosistemi

Kütüphane	Rol
NumPy	Sayısal hesaplama; çok boyutlu diziler, temel; her şey buna dayanır.
Pandas	Etiketli tablo verisi (DataFrame); okuma, temizleme, dönüştürme.
Matplotlib	Temel çizim kütüphanesi; her tür grafik.
Seaborn	Matplotlib üzerine istatistiksel, güzel grafikler.
scikit-learn	Makine öğrenmesi; regresyon, sınıflandırma, kümeleme.
SciPy	İleri bilimsel hesaplama; istatistik, optimizasyon.

1.3 Kurulum ve Ortam

```

1 # Sanal ortam olustur (izole, temiz kurulum)
2 $ python -m venv datascience
3 $ source datascience/bin/activate      # Linux/Mac
4 # datascience\Scripts\activate         # Windows
5
6 # Temel kutuphaneleri kur
7 $ pip install numpy pandas matplotlib seaborn scikit-learn jupyter
8
9 # Jupyter Notebook baslat (interaktif calisma icin ideal)
10 $ jupyter notebook

```

Bilgi

Veri bilimi çalışmaları için **Jupyter Notebook** (veya JupyterLab) neredeyse standarttır: kodu hücre hücre çalıştırır, grafikleri satır içinde gösterir ve keşifsel analizi çok kolaylaştırır. Bu rehberdeki tüm örnekler bir Jupyter hücresinde veya bir .py dosyasında çalıştırılabilir.

2. NumPy: Sayısal Hesaplamanın Temeli

NumPy (Numerical Python), tüm veri bilimi ekosisteminin altında yatan kütüphanedir. Kalbi ndarray (N-boyutlu dizi) nesnesidir: aynı tipteki verileri bellekte ardışık tutan, üzerinde *vektörleştirilmiş* (döngüsüz) işlemler yapılabilen hızlı bir dizi.

2.1 Neden NumPy? Liste vs Dizi

Python listeleri esnektir ama yavaştır: her eleman ayrı bir nesnedir ve döngüler Python yorumlayıcısında çalışır. NumPy dizileri veriyi bitişik bellekte tutar ve işlemleri derlenmiş C koduyla yapar — genelde 10–100 kat daha hızlı.

```

1 import numpy as np
2
3 # Python listesi ile: yavas, dongu gerekir
4 lst = [1, 2, 3, 4]
5 squares = [x**2 for x in lst]      # [1, 4, 9, 16]
6
7 # NumPy ile: hizli, vektorlestirilmis (dongu YOK)
8 arr = np.array([1, 2, 3, 4])
9 squares = arr ** 2                 # array([1, 4, 9, 16])

```

2.2 Dizi Oluşturma

```

1 import numpy as np
2
3 a = np.array([1, 2, 3])           # listeden
4 b = np.zeros((2, 3))              # 2x3 sifir matrisi
5 c = np.ones((3,))                 # [1., 1., 1.]
6 d = np.full((2, 2), 7)            # 7 ile dolu 2x2
7 e = np.eye(3)                     # 3x3 birim matris
8 f = np.arange(0, 10, 2)           # [0 2 4 6 8]
9 g = np.linspace(0, 1, 5)          # [0. 0.25 0.5 0.75 1.]
10 h = np.random.rand(2, 3)          # 2x3 rastgele [0,1)

```

2.3 Dizi Özellikleri: shape, dtype, boyut

Bir dizinin *şekli* (shape) boyutlarını, *dtype*'ı ise sakladığı veri tipini belirtir. Bunları anlamak NumPy'da hataların çoğunu önler.

```

1 a = np.array([[1, 2, 3],
2               [4, 5, 6]])
3 a.shape      # (2, 3) -> 2 satir, 3 sutun
4 a.ndim       # 2      -> boyut sayisi
5 a.size       # 6      -> toplam eleman
6 a.dtype      # dtype('int64')
7
8 # dtype donusumu (bellek/hassasiyet kontrolu icin onemli)
9 b = a.astype(np.float32)
10 c = np.array([1, 2, 3], dtype=np.float64)

```

2.4 Yeniden Şekillendirme (Reshape)

```

1 a = np.arange(12)                # [0 1 2 ... 11]
2 b = a.reshape(3, 4)              # 3x4 matrise cevir
3 c = a.reshape(2, 2, 3)          # 3 boyutlu
4 d = b.reshape(-1)               # duzlestir -> 1B (12,)
5 e = b.T                         # transpoz (4x3)
6 f = b.reshape(4, -1)            # -1: boyutu otomatik hesapla (4x3)

```

Dikkat

reshape verinin bir *görünümünü* (view) döndürür, kopya değil; yani yeni diziyi değiştirmek çoğu zaman orijinali de değiştirir. Bağımsız bir kopya istiyorsan `a.copy()` kullan. Bu view/copy ayrımı, NumPy'da şaşırtıcı hataların en yaygın kaynağıdır.

3. NumPy: İndeksleme, Yayınlama ve İşlemler

3.1 İndeksleme ve Dilimleme

NumPy, Python listelerinden daha güçlü indeksleme sunar: çok boyutlu dilimleme, boole maskeleri ve süslü (fancy) indeksleme.

```

1 a = np.array([[1, 2, 3, 4],
2               [5, 6, 7, 8],
3               [9, 10, 11, 12]])
4
5 a[0, 2]      # 3      -> satir 0, sutun 2
6 a[1]         # [5 6 7 8] -> tum 1. satir
7 a[:, 1]      # [2 6 10]  -> tum satirlarin 1. sutunu
8 a[0:2, 1:3]  # [[2 3] [6 7]] -> alt matris
9 a[-1]       # [9 10 11 12] -> son satir
10
11 # Boole maskesi (kosula uyan elemanlar) -- cok guclu!
12 a[a > 6]     # [7 8 9 10 11 12]
13 a[a % 2 == 0] # cift elemanlar
14
15 # Susulu (fancy) indeksleme
16 a[[0, 2]]    # 0. ve 2. satirlar

```

3.2 Yayınlama (Broadcasting)

Yayınlama, NumPy'nin *farklı şekilli* dizileri birlikte işlemlerini sağlayan kuraldır. Küçük dizi, büyük diziye “yayılarak” uyumlu hale getirilir — bellekte kopya oluşturmada. Bu, veri biliminde standardizasyon gibi işlemlerin temelidir.

```

1 a = np.array([[1, 2, 3],
2               [4, 5, 6]])           # (2, 3)
3
4 a + 10          # her elemana 10 ekle (skaler yayılır)
5 a * 2          # her elemanı 2 ile carp
6
7 row = np.array([10, 20, 30])       # (3,)
8 a + row        # her SATIRA ekle -> [[11 22 33] [14 25 36]]
9
10 col = np.array([[100], [200]])    # (2, 1)
11 a + col       # her SUTUNA ekle -> [[101 102 103] [204 205 206]]
12
13 # Pratik: sutunlari standartlastir (z-skoru)
14 mean = a.mean(axis=0)             # her sutunun ortalamasi
15 std = a.std(axis=0)
16 z = (a - mean) / std              # yayinlama ile

```

Yayınlama kuralı

İki dizi *sondan başa* boyut boyut karşılaştırılır. İki boyut uyumludur eğer: (1) eşitse, ya da (2) biri 1 ise. Örneğin (2,3) ile (3,) uyumludur çünkü (3,), (1,3) gibi ele alınır ve satır boyunca yayılır.

3.3 Eksen (axis) Boyunca İşlemler

Toplama, ortalama gibi indirgeme (reduction) işlemleri bir *eksen* boyunca yapılabilir. `axis=0` sütunlar boyunca (satırları çökertir), `axis=1` satırlar boyunca çalışır. Bu kavram Pandas'ta da aynen geçerlidir.

```

1 a = np.array([[1, 2, 3],
2               [4, 5, 6]])
3
4 a.sum()          # 21 -> tum elemanlar
5 a.sum(axis=0)    # [5 7 9] -> her SUTUN toplami
6 a.sum(axis=1)    # [6 15] -> her SATIR toplami
7 a.mean(axis=0)   # [2.5 3.5 4.5]
8 a.max(axis=1)    # [3 6]
9 a.argmax()       # 5 -> en buyugun (düzleştirilmiş) indeksi
10 np.cumsum(a)    # kumulatif toplam

```

3.4 Doğrusal Cebir

NumPy, makine öğrenmesinin altında yatan matris işlemlerini sağlar.

```

1 A = np.array([[1, 2],
2               [3, 4]])
3 B = np.array([[5, 6],
4               [7, 8]])
5
6 A @ B          # matris carpimi (dot product)
7 A * B          # ELEMAN BAZLI carpim (dikkat: farkli!)
8 np.dot(A, B)   # A @ B ile ayni
9 A.T            # transpoz

```



```

10 np.linalg.inv(A)          # ters matris
11 np.linalg.det(A)          # determinant
12 eigenvalues, eigenvectors = np.linalg.eig(A) # ozdegerler/ozvektorler
13
14 # Dogrusal denklem sistemi cozme: Ax = b
15 b = np.array([1, 2])
16 x = np.linalg.solve(A, b)

```

3.5 Rastgelelik ve Tekrarlanabilirlik

```

1 rng = np.random.default_rng(seed=42)      # tekrarlanabilir (modern API)
2
3 rng.random(3)                             # [0,1) duzgun dagilim
4 rng.integers(1, 7, size=10)                # 1-6 arasi 10 zar atisi
5 rng.normal(0, 1, size=5)                  # standart normal dagilim
6 rng.choice([1, 2, 3], size=4)             # ornekleme
7 shuffled = rng.permutation(np.arange(10)) # karistirma

```

İpucu

Her zaman bir seed (tohum) belirle (`default_rng(42)` veya `np.random.seed(42)`). Böylece analizlerin ve model sonuçların tekrarlanabilir olur — kod her çalıştığında aynı “rastgele” değerleri üretir, bu da hata ayıklamayı ve sonuçları paylaşmayı kolaylaştırır.

4. NumPy Derinlemesine: Veri Tipleri ve Bellek

Yukarıdaki temeller günlük işin %80’ini görür. Ama NumPy’ı gerçekten *anlamak* — neden hızlı olduğunu, neden bazen şaşırtıcı sonuçlar verdiğini — birkaç düşük seviye kavramı bilmeyi gerektirir. Bu bölüm o kavramları bol örnekle açar.

4.1 Veri Tipleri (dtype) Derinlemesine

Bir NumPy dizisindeki *tüm* elemanlar aynı tiptedir ve her tip bellekte sabit sayıda bayt kaplar. Doğru dtype seçmek hem bellek hem hız kazandırır: 100 milyonluk bir diziyi `int64` yerine `int8` tutmak belleği 8 kat azaltır.

```

1 import numpy as np
2
3 # Tam sayılar: int8/16/32/64 (isaretli), uint8/... (isaretsiz)
4 a = np.array([1, 2, 3], dtype=np.int8) # her eleman 1 bayt
5 a.itemsize                             # 1 -> eleman başına bayt
6 a.nbytes                               # 3 -> toplam bayt
7
8 # Ondalıklı: float16/32/64
9 b = np.array([1.5, 2.5], dtype=np.float32)
10
11 # Bool ve metin
12 c = np.array([True, False, True])      # dtype('bool')
13 d = np.array(['al', 'sat'])             # dtype('<U3') sabit uzunluk!
14
15 # Karisik listeden olusturmada NumPy ortak tip secer (upcasting)
16 np.array([1, 2, 3.0]).dtype             # float64 (int -> float'a yukselir)
17 np.array([1, 'x']).dtype                # '<U21' (hepsi metne dondu!)

```

Dikkat

Tam sayı taşması (overflow) sessizdir! Sabit boyutlu tipler sınırı aşınca döner (wrap-around) — hata vermez. `int8` en fazla 127 tutar; `np.int8(127) + 1` sonucu -128 olur. Büyük değerlerle çalışırken `int64` veya `float64` kullan.

```

1 x = np.array([127], dtype=np.int8)
2 x + 1          # array([-128])  <- SESSİZ TASMA!
3
4 # astype ile güvenli dönüşüm
5 big = x.astype(np.int64) + 1    # array([128])  doğru
6
7 # float -> int dönüşümünde ondalık ATILIR (yuvarlanmaz)
8 np.array([1.9, 2.1, -1.9]).astype(int)    # array([ 1,  2, -1])
9 np.round([1.9, 2.1]).astype(int)         # array([2,  2])  önce yuvarla

```

4.2 Görünüm (View) mü, Kopya (Copy) mı?

Bu, NumPy'daki en sık “neden değişti?!” hatasının kaynağıdır. **Dilimleme (slicing) bir görünüm döndürür** — aynı belleğe bakan yeni bir pencere. Görünümü değiştirmek orijinali de değiştirir. **Süslü/boole indeksleme ise kopya döndürür.**

```

1 a = np.arange(10)          # [0 1 2 3 4 5 6 7 8 9]
2
3 # DILIM = GÖRÜNÜM (aynı bellek)
4 s = a[2:5]                 # [2 3 4]
5 s[0] = 999
6 a                          # [0 1 999 3 4 5 6 7 8 9]  <- a DEĞİŞTİ!
7
8 # .copy() ile bağımsız kopya
9 s2 = a[2:5].copy()
10 s2[0] = -1                # a etkilenmez
11
12 # Bir dizinin görünüm mü olduğunu kontrol et
13 s.base is a                # True -> s, a'nın görünümü
14 s2.base is a               # False -> bağımsız kopya
15
16 # SÜSLÜ/BOOLE indeksleme her zaman KOPYA doner
17 f = a[[1, 3, 5]]           # kopya
18 m = a[a > 5]               # kopya
19 f[0] = 0                   # a'yi ETKİLEMEZ

```

Neden önemli?

Fonksiyona dizi geçirip içinde dilimleyerek değiştirirsen, çağırının verisini farkında olmadan bozabilirsin. Kural: veriyi değiştirecek bir fonksiyon yazıyorsan ya baştan `arr.copy()` al, ya da davranışı belgele. Tersine, büyük veride gereksiz `.copy()` bellek israfıdır — görünüm bedavadır.

4.3 Bir Diziyi Yerinde Değiştirmek

Görünümler, koşullu güncellemeyi çok verimli kılar: yeni dizi oluşturmadan.

```

1 a = np.array([-3, 5, -1, 8, -7, 2])
2
3 # Boole maske ile YERİNDE atama (negatifleri sıfırla)
4 a[a < 0] = 0              # [0 5 0 8 0 2]
5

```

```

6 # np.clip: degerleri bir araliga sikistir
7 np.clip(a, 0, 5) # [0 5 0 5 0 2]
8
9 # np.where: kosullu SECIM (if-else vektorlestirilmis)
10 b = np.array([10, -2, 7, -5])
11 np.where(b > 0, b, 0) # pozitifse kendisi, degilse 0
12 np.where(b > 0, 'arti', 'eksi') # [arti eksi arti eksi]
13
14 # np.select: cok kosullu (elif zinciri gibi)
15 notlar = np.array([95, 82, 60, 45])
16 kosullar = [notlar >= 90, notlar >= 70, notlar >= 50]
17 sonuc = np.select(kosullar, ['A', 'B', 'C'], default='F')
18 # ['A' 'B' 'C' 'F']

```

5. NumPy Derinlemesine: Vektörizasyon ve Performans

5.1 Evrensel Fonksiyonlar (ufunc) ve Vektörizasyon

NumPy'nin gücü *vektörizasyondan* gelir: bir işlemi tüm diziye tek seferde, derlenmiş C döngüsüyle uygulamak. Python for döngüsünden kaçındığın her yerde büyük hız kazanırsın. `np.sin`, `+`, `**` gibi eleman bazlı çalışan fonksiyonlara *ufunc* denir.

```

1 x = np.linspace(0, 2*np.pi, 1_000_000)
2
3 # YAVAS: Python dongusu (kacin!)
4 # y = np.array([np.sin(v) for v in x])
5
6 # HIZLI: vektorlestirilmis (10-100x daha hizli)
7 y = np.sin(x)
8
9 # Eleman bazli ufunc'lar zincirlenebilir
10 z = np.exp(-x) * np.cos(x) + 1
11
12 # Matematiksel yardimcilar
13 np.sqrt([1, 4, 9]) # [1. 2. 3.]
14 np.log(np.e) # 1.0
15 np.abs([-2, 3, -5]) # [2 3 5]
16 np.maximum([1, 5], [4, 2]) # [4 5] eleman bazli maks

```

Örnek: Vektörizasyonun Gücü

İki nokta kümesi arasındaki tüm mesafeleri tek satırda, döngüsüz hesaplamak — yayınlama + ufunc birleşimi:

```

1 noktalar = np.array([[0, 0], [3, 4], [6, 8]]) # (3, 2)
2
3 # Her çiftin oklit mesafesi (yayınlama ile)
4 fark = noktalar[:, None, :] - noktalar[None, :, :] # (3, 3, 2)
5 mesafe = np.sqrt((fark ** 2).sum(axis=2)) # (3, 3)
6 # mesafe[i, j] = i. ve j. nokta arasi uzaklik

```

Bilgi

`np.vectorize` bir Python fonksiyonunu diziye uygular ama içeride gerçek bir döngü çalıştırır — hız kazandırmaz, sadece kolaylık sağlar. Gerçek hız için işlemi NumPy `ufunc`'larıyla ifade et.

5.2 Dizi Birleştirme ve Bölme

Veriyi yan yana veya üst üste eklemek, ML'de özellik matrisleri kurmanın temelidir.

```

1 a = np.array([[1, 2], [3, 4]])
2 b = np.array([[5, 6], [7, 8]])
3
4 np.concatenate([a, b], axis=0) # dikey (satir ekle) -> (4, 2)
5 np.concatenate([a, b], axis=1) # yatay (sutun ekle) -> (2, 4)
6 np.vstack([a, b])             # dikey (v = vertical)
7 np.hstack([a, b])             # yatay (h = horizontal)
8
9 # stack: YENI bir eksen ekler
10 np.stack([a, b])              # (2, 2, 2) -> ustuste yeni katman
11
12 # Bir sutunu/satiri diziye ekleme icin newaxis
13 v = np.array([1, 2])          # (2,)
14 v[:, None]                    # (2, 1) sutun vektörü
15 v[None, :]                    # (1, 2) satir vektörü
16
17 # Bolme
18 c = np.arange(12).reshape(3, 4)
19 np.split(c, 2, axis=1)        # 4 sutunu 2 esit parcaya ([::2],[::2:])

```

5.3 Sıralama, Arama ve Faydalı İşlemler

```

1 a = np.array([3, 1, 4, 1, 5, 9, 2, 6])
2
3 np.sort(a)                     # [1 1 2 3 4 5 6 9] (kopya doner)
4 np.argsort(a)                  # siralayan INDEKSLER -> [1 3 6 0 2 7 4 5]
5 a.argmax(), a.argmin()         # (5, 1) en buyuk/kucuk indeksi
6
7 np.unique(a)                   # [1 2 3 4 5 6 9] tekrarsiz
8 np.unique(a, return_counts=True) # degerler + kac kez geldikleri
9
10 # En buyuk 3 degeri getir (argsort ile)
11 a[np.argsort(a)[-3:]]         # [5 6 9]
12
13 # Kosula uyan elemanların INDEKSLERİ
14 np.nonzero(a > 3)              # (array([2, 4, 5, 7]),)
15
16 # Kumulatif ve istatistik
17 a.cumsum()                     # kumulatif toplam
18 np.percentile(a, [25, 50, 75]) # ceyrekler
19 np.isnan(np.array([1, np.nan])) # [False True] -> eksik deger bulma

```

5.4 Neden Bu Kadar Hızlı? Bellek Düzeni

NumPy hızının sırrı: veri bellekte *tek bir bitişik blokta* durur ve işlemler CPU'ya yakın, önbellek dostu şekilde çalışır. `strides`, bir ekseninde bir adım atmak için kaç bayt atlanacağını söyler; `reshape` ve `transpose` genelde veriyi kopyalamaz, sadece bu “adım” bilgisini değiştirir.

```
1 a = np.arange(12).reshape(3, 4)
2 a.strides      # (32, 8) -> satir atla=32 bayt, sutun atla=8 bayt
3 a.flags['C_CONTIGUOUS'] # True -> satir-öncelikli (C düzeni)
4
5 # Transpoz veriyi KOPYALAMAZ, sadece strides'i takas eder (bedava)
6 a.T.strides    # (8, 32)
7
8 # .ravel() bitisikse görünüm, değilse kopya doner
9 a.ravel()      # düzleştir (mümkünse görünüm)
10 np.ascontiguousarray(a.T) # gerekince bellekte gerçekten sırala
```

İpucu

Pratik özet: (1) **Döngü yazma** — işlemi tüm diziye uygula. (2) **Doğru dtype seç** — gereksiz float64 bellek yer. (3) **Görünüm/kopya farkını bil** — beklenmedik değişiklikleri önler. (4) Aradığın işlem muhtemelen zaten np. altında vardır — önce ara, sonra elle yazma.

KISIM 2

Pandas ile Veri İşleme

6. Pandas Temelleri: Series ve DataFrame

Pandas, etiketli ve tablo biçimli veriyle çalışmak için tasarlanmış kütüphanedir; adını “panel data”dan alır. İki temel yapısı vardır: **Series** (etiketli tek boyutlu dizi) ve **DataFrame** (etiketli satır ve sütunlardan oluşan iki boyutlu tablo — bir Excel sayfası gibi). Gerçek veri biliminin büyük kısmı Pandas’ta geçer.

6.1 Series: Etiketli Tek Boyutlu Veri

Series, bir NumPy dizisi ile onu tanımlayan bir *indeksin* (etiketler) birleşimidir. Sözlük ve dizinin özelliklerini birleştirir.

```
1 import pandas as pd
2
3 s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
4 s['b']          # 20    -> etiketle erisim
5 s[1]           # 20    -> konumla erisim
6 s[s > 15]      # b,c,d -> boole filtreleme
7 s * 2          # her elemani ikiye katla (vektorlestirilmis)
8 s.mean()       # 25.0
9
10 # Sozlukten Series
11 population = pd.Series({'Ankara': 5.7, 'Izmir': 4.4, 'Istanbul': 15.8})
```

6.2 DataFrame: Tablo Verisi

DataFrame, her biri bir Series olan sütunların birleşimidir. Veri biliminin ana çalışma nesnesidir.

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     'name':    ['Ali', 'Ayse', 'Can', 'Deniz'],
5     'age':     [25, 30, 35, 28],
6     'city':    ['Ankara', 'Izmir', 'Ankara', 'Istanbul'],
7     'salary':  [5000, 7000, 6500, 8000]
8 })
9
10 df.head()      # ilk 5 satir
11 df.tail(2)     # son 2 satir
12 df.shape       # (4, 4)
13 df.columns     # sutun adlari
```

```

14 df.dtypes          # her sütunun tipi
15 df.info()          # genel özet (tip, bellek, eksik)
16 df.describe()      # sayısal sütunların istatistiği

```

6.3 İlk Bakış: describe ve info

Yeni bir veri setiyle karşılaştığında ilk yapılacak, onu “tanımaktır”. Bu iki metot bir veri setinin röntgenini çeker.

Metot	Ne gösterir?
<code>df.info()</code>	Satır sayısı, sütun tipleri, eksik olmayan değer sayısı, bellek.
<code>df.describe()</code>	Sayısal sütunlar için sayı, ortalama, std, min, çeyrekler, max.
<code>df.nunique()</code>	Her sütundaki benzersiz değer sayısı.
<code>df['s'].value_counts()</code>	Bir sütundaki her değer kaç kez geçtiği.

7. Veri Okuma ve Yazma

Pandas, çok çeşitli kaynaklardan veri okuyabilir. Gerçek projelerde en sık CSV ve Excel kullanılır, ama veritabanları ve API'ler de yaygındır.

```

1 import pandas as pd
2
3 # --- OKUMA ---
4 df = pd.read_csv('veri.csv')          # CSV
5 df = pd.read_csv('veri.csv', sep=';', # özel ayırıcı
6                      encoding='utf-8', na_values=['NA', '?'])
7 df = pd.read_excel('veri.xlsx', sheet_name='Sayfa1')
8 df = pd.read_json('veri.json')
9 df = pd.read_sql('SELECT * FROM table', connection) # veritabanı
10
11 # --- YAZMA ---
12 df.to_csv('cikti.csv', index=False)    # index=False: satır no yazma
13 df.to_excel('cikti.xlsx', index=False)
14 df.to_json('cikti.json', orient='records')

```

Bilgi

`read_csv` onlarca parametre alır. En sık kullanılanlar: `sep` (ayırıcı), `header` (başlık satırı), `index_col` (indeks olarak kullanılacak sütun), `usecols` (sadece belirli sütunlar), `parse_dates` (tarih sütunlarını otomatik dönüştür), `nrows` (büyük dosyalarda ilk N satır). Büyük dosyaları `chunks` ile parça parça da okuyabilirsiniz.

8. Veri Seçme ve Filtreleme

DataFrame'den veri seçmenin birden çok yolu vardır. En önemli ayırım: `loc` (etiket tabanlı) ve `iloc` (konum/tam sayı tabanlı).

8.1 Sütun ve Satır Seçme

```

1 df['age']                                # tek sutun -> Series
2 df[['name', 'age']]                     # birden cok sutun -> DataFrame
3
4 # loc: ETIKET tabanlı (satır etiketi, sütun adı)
5 df.loc[0]                              # 0 etiketli satır
6 df.loc[0, 'name']                       # tek hücre
7 df.loc[0:2, ['name', 'age']]            # satır 0-2 (DAHİL), belirli sütunlar
8
9 # iloc: KONUM tabanlı (tam sayı indeksleri)
10 df.iloc[0]                             # ilk satır
11 df.iloc[0:2]                           # ilk 2 satır (2 HARİC, Python dilimi gibi)
12 df.iloc[-1]                            # son satır
13 df.iloc[0:2, 0:2]                       # ilk 2 satır, ilk 2 sütun

```

Dikkat

loc ile dilimlemede son sınır **dahildir** (`df.loc[0:2]` → 0,1,2), ama iloc ve normal Python dilimlemesinde son sınır **hariştir** (`df.iloc[0:2]` → 0,1). Bu, yeni başlayanların en sık yaptığı hatalardan biridir.

8.2 Boole Filtreleme (En Önemli Beceri)

Koşullara göre satır seçmek veri analizinin bel kemiğidir. Koşullar boole maskesi üretir; bu maske DataFrame'i filtreler.

```

1 # Tek koşul
2 df[df['age'] > 28]
3
4 # Birden çok koşul: & (ve), | (veya), ~ (değil) -- PARANTEZ şart!
5 df[(df['age'] > 25) & (df['city'] == 'Ankara')]
6 df[(df['salary'] > 6000) | (df['age'] < 26)]
7
8 # isin: bir listedeki değerler
9 df[df['city'].isin(['Ankara', 'Izmir'])]
10
11 # between: aralık
12 df[df['age'].between(25, 32)]
13
14 # String içeren
15 df[df['name'].str.startswith('A')]
16 df[df['name'].str.contains('e')]
17
18 # query: SQL benzeri okunabilir sözdizimi
19 df.query('age > 28 and city == "Ankara"')

```

8.3 Yeni Sütun Oluşturma ve Dönüştürme

```

1 df['annual_salary'] = df['salary'] * 12      # vektörleştirilmiş işlem
2 df['age_group'] = df['age'].apply(
3     lambda a: 'young' if a < 30 else 'adult')
4
5 # np.where ile koşullu sütun
6 import numpy as np
7 df['level'] = np.where(df['salary'] > 6500, 'high', 'normal')
8

```



```

9 # map ile deger esleme
10 df['city_code'] = df['city'].map({'Ankara': 6, 'Izmir': 35, 'Istanbul': 34})
11
12 # Sutun/satir silme
13 df = df.drop(columns=['city_code'])
14 df = df.drop(index=[0])

```

9. Veri Temizleme

Gerçek veri asla temiz gelmez: eksik değerler, tekrarlar, yanlış tipler ve tutarsızlıklar içerir. Zamanının çoğunu buna harcarsın. Temiz veri, iyi analizin ön koşuludur.

9.1 Eksik Veri (NaN) ile Başa Çıkma

```

1 # Eksikleri tespit et
2 df.isnull().sum() # her sutundaki eksik sayisi
3 df.isnull().sum().sum() # toplam eksik
4
5 # Eksik satirlari/sutunlari SIL
6 df.dropna() # eksik iceren satirlari at
7 df.dropna(axis=1) # eksik iceren sutunlari at
8 df.dropna(subset=['salary']) # sadece belirli sutunda eksik olanlari at
9 df.dropna(thresh=3) # en az 3 dolu deger olan satirlari tut
10
11 # Eksikleri DOLDUR (imputation)
12 df['salary'].fillna(0) # sabitle
13 df['salary'].fillna(df['salary'].mean()) # ortalama ile (sik)
14 df['salary'].fillna(df['salary'].median()) # medyan ile (aykiri degere dayanikli)
15 df['city'].fillna(df['city'].mode()[0]) # en sik deger (kategorik)
16 df.fillna(method='ffill') # onceki degerle (zaman serisi)

```

Silmek mi doldurmak mı?

Eksik veri azsa (%5'ten az) satırları silmek güvenlidir. Ama çoksa silmek veriyi ziyan eder — doldurmak (imputation) gerekir. Sayısal sütunlarda *medyan* (aykırı değerlere dayanıklı), kategoriklerde *mod* sık kullanılır. Neden eksik olduklarını da düşün: rastgele mi, yoksa sistematik mi?

9.2 Tekrarlar ve Tip Dönüşümleri

```

1 # Tekrarlar
2 df.duplicated().sum() # kac tekrar satir var
3 df = df.drop_duplicates() # tekrarlar sil
4 df = df.drop_duplicates(subset=['name']) # belirli sutuna gore
5
6 # Tip donusumu
7 df['age'] = df['age'].astype(int)
8 df['date'] = pd.to_datetime(df['date'])
9 df['salary'] = pd.to_numeric(df['salary'], errors='coerce') # hatalari NaN yap
10
11 # String temizleme
12 df['name'] = df['name'].str.strip() # bosluklari kirp
13 df['city'] = df['city'].str.lower() # kucuk harf
14 df['name'] = df['name'].str.replace('.', '', regex=False)

```

```

15
16 # Sütun yeniden adlandırma
17 df = df.rename(columns={'name': 'full_name', 'age': 'age_years'})

```

10. Gruplama ve Toplulaştırma (GroupBy)

groupby, verinin belki de en güçlü aracıdır: veriyi bir veya daha çok sütuna göre gruplara böler, her gruba bir işlem (ortalama, toplam, sayım...) uygular ve sonuçları birleştirir. Buna *böl-uygula-birleştir* (split-apply-combine) denir.

```

1 # Şehre göre ortalama maaş
2 df.groupby('city')['salary'].mean()
3
4 # Birden çok istatistik
5 df.groupby('city')['salary'].agg(['mean', 'min', 'max', 'count'])
6
7 # Birden çok sütuna göre gruplama
8 df.groupby(['city', 'age_group'])['salary'].mean()
9
10 # Her sütuna farklı fonksiyon
11 df.groupby('city').agg({
12     'salary': 'mean',
13     'age': 'max',
14     'name': 'count'
15 })
16
17 # Özel fonksiyon
18 df.groupby('city')['salary'].agg(lambda x: x.max() - x.min()) # aralık

```

10.1 Pivot Tablolar

Pivot tablo, veriyi iki boyutlu bir özet tabloya dönüştürür — Excel'deki pivot tablonun aynısı. Bir sütunu satırlara, diğerini sütunlara yayar.

```

1 pd.pivot_table(df,
2     values='salary',      # özetlenecek değer
3     index='city',         # satırlar
4     columns='age_group',  # sütunlar
5     aggfunc='mean',       # toplulaştırma fonksiyonu
6     fill_value=0)         # boş hücreleri doldur
7
8 # crosstab: sayım için pratik
9 pd.crosstab(df['city'], df['age_group'])

```

11. Veri Birleştirme

Gerçek projelerde veri genelde birden çok tablodan gelir ve birleştirilmesi gerekir. Pandas bunun için merge (SQL join gibi), concat (uç uca ekleme) ve join sunar.

11.1 merge: Anahtar Üzerinden Birleştirme

```

1 employees = pd.DataFrame({

```

```

2      'id': [1, 2, 3], 'name': ['Ali', 'Ayse', 'Can'], 'dept_id': [10, 20, 10])
3  departments = pd.DataFrame({
4      'dept_id': [10, 20, 30], 'dept': ['Software', 'Sales', 'HR']})
5
6  # Ortak sutun (dept_id) üzerinden birlestir
7  pd.merge(employees, departments, on='dept_id', how='inner')

```

how

Hangi satırlar tutulur?

inner Sadece her iki tabloda da eşleşen anahtarlar (kesişim).

left Sol tablonun tümü + sağdan eşleşenler.

right Sağ tablonun tümü + soldan eşleşenler.

outer Her iki tablonun tümü (birleşim); eşleşmeyenler NaN.

11.2 concat: Uç Uca Ekleme

```

1  # Satir bazli birlestirme (alt alta) -- ayni sutunlar
2  pd.concat([df1, df2], axis=0, ignore_index=True)
3
4  # Sutun bazli birlestirme (yan yana) -- ayni satirlar
5  pd.concat([df1, df2], axis=1)

```

12. Zaman Serileri

Zaman serisi verisi (tarih/saat damgalı ölçümler) veri biliminde çok yaygındır: borsa, hava, satış... Pandas bunun için güçlü araçlar sunar.

```

1  # Tarih indeksi olustur
2  df['date'] = pd.to_datetime(df['date'])
3  df = df.set_index('date')
4
5  # Tarih bileşenlerine erisim
6  df.index.year
7  df.index.month
8  df.index.dayofweek      # 0=Pazartesi
9
10 # Tarihe gore dilimleme
11 df['2026']               # 2026 yilinin tumu
12 df['2026-01':'2026-03'] # ilk uc ay
13
14 # Yeniden ornekleme (resample): gunluk -> aylık
15 df['sales'].resample('M').sum()      # aylık toplam
16 df['sales'].resample('W').mean()     # haftalık ortalama
17
18 # Kayan pencere (hareketli ortalama) -- trend icin
19 df['mean_7d'] = df['sales'].rolling(window=7).mean()
20
21 # Kaydırma ve degisim
22 df['previous'] = df['sales'].shift(1)      # bir onceki deger
23 df['change'] = df['sales'].pct_change()    # yuzde degisim

```

İpucu

Zaman serilerinde `resample` (zaman frekansını değiştirme) ve `rolling` (hareketli pencere) en çok kullanılan iki araçtır. Hareketli ortalama, gürültülü veride *trendi* görmenin en basit yoludur; günlük dalgalanmaları yumuşatır.

KISIM 3

Veri Görselleştirme

13. Matplotlib: Temel Çizim

Matplotlib, Python'ın temel çizim kütüphanesidir; diğer görselleştirme kütüphaneleri (Seaborn, Pandas'ın çizimi) onun üzerine kuruludur. Güçlü ve esnektir ama biraz ayrıntılıdır. İyi bir görselleştirme, bir tablodan çok daha hızlı içgörü verir — “bir resim bin satıra bedeldir”.

13.1 Anatomi: Figure ve Axes

Matplotlib'de bir grafik iki katmandan oluşur: **Figure** (tüm tuval) ve **Axes** (üzerindeki tek bir grafik alanı — eksenler, veri burada çizilir). Modern ve önerilen kullanım, bu ikisini açıkça oluşturan *nesne yönelimli* arayüzdür.

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 x = np.linspace(0, 10, 100)
5 y = np.sin(x)
6
7 fig, ax = plt.subplots(figsize=(8, 4))    # Figure + tek Axes
8 ax.plot(x, y, label='sin(x)', color='navy', linewidth=2)
9 ax.set_title('Sinus Eğrisi')
10 ax.set_xlabel('x')
11 ax.set_ylabel('sin(x)')
12 ax.legend()
13 ax.grid(True, alpha=0.3)
14 plt.savefig('grafik.png', dpi=150, bbox_inches='tight') # dosyaya kaydet
15 plt.show()
```

13.2 Temel Grafik Türleri

Doğru grafik türünü seçmek, verinin doğasına bağlıdır.

Grafik**Ne zaman kullanılır?**

plot	Çizgi grafik; trend, zaman serisi.
scatter	Dağılım; iki sayısal değişken arası ilişki.
bar	Çubuk; kategoriler arası karşılaştırma.
hist	Histogram; tek değişkenin dağılımı.
boxplot	Kutu grafiği; dağılım + aykırı değerler.
pie	Pasta; bütünün parçaları (dikkatli kullan).

```

1 fig, ax = plt.subplots()
2
3 ax.scatter(df['age'], df['salary'])           # dagilim
4 ax.bar(df['name'], df['salary'])              # cubuk
5 ax.hist(df['salary'], bins=20)                # histogram
6 ax.boxplot([group1, group2], labels=['A', 'B']) # kutu grafigi
7 ax.pie(values, labels=labels, autopct='%1.1f%%') # pasta

```

13.3 Birden Çok Alt Grafik (Subplots)

Genelde birden çok grafiği yan yana göstermek isteriz. `plt.subplots` bir ızgara oluşturur.

```

1 fig, axes = plt.subplots(2, 2, figsize=(10, 8)) # 2x2 ızgara
2
3 axes[0, 0].plot(x, np.sin(x)); axes[0, 0].set_title('sin')
4 axes[0, 1].plot(x, np.cos(x)); axes[0, 1].set_title('cos')
5 axes[1, 0].hist(df['salary'])
6 axes[1, 1].scatter(df['age'], df['salary'])
7
8 fig.suptitle('Panel Grafikler', fontsize=14)
9 fig.tight_layout() # cakismalari onle
10 plt.show()

```

13.4 Stil ve Özelleştirme

```

1 plt.style.use('seaborn-v0_8') # hazır stil temasi
2
3 ax.plot(x, y,
4         color='#2E6DA4', linestyle='--', linewidth=2,
5         marker='o', markersize=4, alpha=0.8, label='veri')
6
7 ax.set_xlim(0, 10) # eksen sinirlari
8 ax.set_ylim(-1.5, 1.5)
9 ax.set_xticks([0, 2, 4, 6, 8, 10]) # isaret konumlari
10 ax.axhline(0, color='gray', linewidth=0.8) # yatay referans cizgisi
11 ax.annotate('tepe', xy=(1.57, 1), xytext=(3, 1.2),
12            arrowprops=dict(arrowstyle='->')) # ok ile aciklama

```

Bilgi

pyplot vs nesne-yönelimli: `plt.plot()` gibi durum tabanlı (stateful) arayüz hızlı denemeler için pratiktir, ama karmaşık grafiklerde kafa karıştırır. Ciddi işte her zaman `fig, ax = plt.subplots()` ile başlayıp `ax` üzerinden çalış — daha okunaklı ve kontrol edilebilirdir.

14. Matplotlib Derinlemesine: Grafik Türleri Kataloğu

Yukarıdaki temeller grafik iskeletini kurar. Şimdi her grafik türünü *çalışan, tam örneklerle* inceleyelim. Her örnek kendi başına çalışır — kopyala, çalıştır, değiştirerek öğren.

14.1 Çizgi Grafik: Birden Çok Seri ve Stil

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 x = np.linspace(0, 10, 200)
5 fig, ax = plt.subplots(figsize=(8, 4))
6
7 ax.plot(x, np.sin(x), color='#2E6DA4', lw=2, label='sin')
8 ax.plot(x, np.cos(x), color='#D9701E', lw=2, ls='--', label='cos')
9 ax.plot(x, np.sin(x)*np.exp(-x/5), color='#2E7D32', lw=2,
10         ls=':', label='sonumlu')
11
12 ax.fill_between(x, np.sin(x), 0, alpha=0.1, color='#2E6DA4') # alan doldur
13 ax.set(title='Çizgi Grafikleri', xlabel='x', ylabel='y')
14 ax.legend(loc='upper right', frameon=True)
15 ax.grid(True, alpha=0.3)
16 plt.show()
```

14.2 Dağılım Grafiği: Renk ve Boyutla Kodlama

Bir dağılım grafiğinde 4 boyutu aynı anda gösterebilirsiniz: x, y, renk (c) ve nokta boyutu (s).

```
1 rng = np.random.default_rng(0)
2 x = rng.random(80); y = rng.random(80)
3 renk = rng.random(80) # 3. boyut -> renk
4 boyut = rng.random(80) * 400 # 4. boyut -> alan
5
6 fig, ax = plt.subplots(figsize=(7, 5))
7 sc = ax.scatter(x, y, c=renk, s=boyut, cmap='viridis',
8               alpha=0.7, edgecolors='white', linewidth=0.5)
9 fig.colorbar(sc, ax=ax, label='renk degeri') # renk olcegi
10 ax.set(title='Dağılım: 4 boyut', xlabel='x', ylabel='y')
11 plt.show()
```

14.3 Çubuk Grafik: Gruplu ve Yığılmış

```
1 kategoriler = ['Ocak', 'Subat', 'Mart', 'Nisan']
2 urun_a = [20, 34, 30, 35]
3 urun_b = [25, 32, 34, 20]
4 x = np.arange(len(kategoriler))
5 w = 0.35
6
7 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))
8
9 # Gruplu (yan yana)
10 ax1.bar(x - w/2, urun_a, w, label='A', color='#2E6DA4')
11 ax1.bar(x + w/2, urun_b, w, label='B', color='#D9701E')
12 ax1.set_xticks(x); ax1.set_xticklabels(kategoriler)
13 ax1.set_title('Gruplu Çubuk'); ax1.legend()
14
```

```

15 # Yigilmis (ust uste)
16 ax2.bar(kategoriler, urun_a, label='A', color='#2E6DA4')
17 ax2.bar(kategoriler, urun_b, bottom=urun_a, label='B', color='#D9701E')
18 ax2.set_title('Yigilmis Cubuk'); ax2.legend()
19
20 fig.tight_layout(); plt.show()

```

14.4 Histogram ve Yoğunluk: Dağılımı Görmek

```

1 veri1 = np.random.default_rng(1).normal(60, 10, 1000)
2 veri2 = np.random.default_rng(2).normal(75, 8, 1000)
3
4 fig, ax = plt.subplots(figsize=(8, 4))
5 ax.hist(veri1, bins=30, alpha=0.6, color='#2E6DA4', label='Grup 1',
6         density=True) # density: alan=1
7 ax.hist(veri2, bins=30, alpha=0.6, color='#D9701E', label='Grup 2',
8         density=True)
9 ax.axvline(veri1.mean(), color='#2E6DA4', ls='--', lw=2) # ortalama cizgisi
10 ax.axvline(veri2.mean(), color='#D9701E', ls='--', lw=2)
11 ax.set(title='İki Grubun Dağılımı', xlabel='puan', ylabel='yoğunluk')
12 ax.legend(); plt.show()

```

14.5 Kutu ve Keman: Grupları Karşılaştırmak

```

1 gruplar = [np.random.default_rng(i).normal(50+i*10, 8, 100)
2            for i in range(4)]
3
4 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))
5 ax1.boxplot(gruplar, tick_labels=['A', 'B', 'C', 'D'])
6 ax1.set_title('Kutu Grafiği (medyan, çeyrek, aykırı)')
7
8 parts = ax2.violinplot(gruplar, showmeans=True) # keman: dağılım şekli
9 ax2.set_xticks([1, 2, 3, 4]); ax2.set_xticklabels(['A', 'B', 'C', 'D'])
10 ax2.set_title('Keman Grafiği')
11 fig.tight_layout(); plt.show()

```

15. Matplotlib Derinlemesine: Eksen, Yazı ve Kaydetme

15.1 Eksenleri Tam Kontrol: Sınırlar, İşaretler, Ölçek

```

1 import matplotlib.ticker as mticker
2
3 fig, ax = plt.subplots(figsize=(8, 4))
4 x = np.linspace(1, 100, 100)
5 ax.plot(x, x**2)
6
7 ax.set_xlim(0, 100); ax.set_ylim(1, 1e4)
8 ax.set_yscale('log') # logaritmik eksen
9 ax.set_xticks([0, 25, 50, 75, 100])
10 ax.set_xticklabels(['0', 'çeyrek', 'yari', '3/4', 'tam'], rotation=45)
11
12 # Ekseni yüzde/para birimi olarak biçimle
13 ax.yaxis.set_major_formatter(mticker.FuncFormatter(

```



```

14     lambda v, _: f'{v:,.0f} TL'))
15 ax.tick_params(axis='both', labelsize=9)
16 plt.show()

```

15.2 Metin, Ok ve Vurgulama (Annotation)

Bir grafiği anlatan şey çoğu zaman üzerindeki açıklamalardır: bir zirveyi işaretlemek, bir eşiği göstermek.

```

1 x = np.linspace(0, 10, 100)
2 y = np.sin(x)
3 fig, ax = plt.subplots(figsize=(8, 4))
4 ax.plot(x, y, color='#2E6DA4', lw=2)
5
6 # Bir noktaya ok ile aciklama
7 tepe_x = np.pi/2
8 ax.annotate('en yuksek nokta',
9             xy=(tepe_x, 1), xytext=(4, 1.3),
10            arrowprops=dict(arrowstyle='->', color='#C0392B'),
11            fontsize=11, color='#C0392B')
12
13 # Serbest metin ve referans çizgileri
14 ax.text(7, -0.8, r'$y=\sin(x)$', fontsize=12) # LaTeX matematik
15 ax.axhline(0, color='gray', lw=0.8)
16 ax.axvspan(3, 6, alpha=0.1, color='#D9701E') # dikey bant vurgusu
17 ax.set(title='Aciklamali Grafik', ylim=(-1.5, 1.6))
18 plt.show()

```

15.3 İkiz Eksen (Twin Axes): Farklı Birimler

Sıcaklık ve satış gibi farklı ölçekli iki seriyi tek grafikte göstermek için.

```

1 aylar = np.arange(1, 13)
2 sicaklik = [5, 6, 10, 15, 20, 25, 28, 27, 22, 16, 10, 6]
3 satis = [200, 180, 220, 300, 450, 600, 700, 680, 500, 350, 250, 210]
4
5 fig, ax1 = plt.subplots(figsize=(8, 4))
6 ax1.plot(aylar, sicaklik, 'o-', color='#C0392B')
7 ax1.set_xlabel('ay'); ax1.set_ylabel('sicaklik (C)', color='#C0392B')
8 ax1.tick_params(axis='y', labelcolor='#C0392B')
9
10 ax2 = ax1.twinx() # ayni x, farkli y eksenini
11 ax2.bar(aylar, satis, alpha=0.3, color='#2E6DA4')
12 ax2.set_ylabel('satis (adet)', color='#2E6DA4')
13 ax2.tick_params(axis='y', labelcolor='#2E6DA4')
14 fig.suptitle('Sicaklik vs Satis')
15 plt.show()

```

15.4 Karmaşık Yerleşim ve Yüksek Kaliteli Kaydetme

```

1 # Farkli boyutlarda paneller icin mosaic (en pratik yontem)
2 fig, axd = plt.subplot_mosaic(
3     [['ust', 'ust'],
4     ['sol', 'sag']], figsize=(9, 6))
5 axd['ust'].plot(np.random.default_rng(0).random(50).cumsum())
6 axd['sol'].hist(np.random.default_rng(1).normal(size=200))
7 axd['sag'].scatter(np.random.default_rng(2).random(50),

```

```

8         np.random.default_rng(3).random(50))
9 for ad, ax in axd.items():
10     ax.set_title(ad)
11 fig.tight_layout()
12
13 # Yayına uygun kaydetme
14 fig.savefig('rapor.png', dpi=300, bbox_inches='tight') # yuksek cozunurluk
15 fig.savefig('rapor.pdf', bbox_inches='tight')         # vektorel (olcek bozulmaz)
16 fig.savefig('rapor.svg')                             # web/duzenlenebilir

```

Bilgi

Kaydetme ipuçları: Raporlar için dpi=300 kullan; sunum ve web için bbox_inches='tight' kenar boşluklarını kırpar. Ölçeklendiğinde bulanıklaşmaması gereken grafikleri (dergi, poster) .pdf veya .svg olarak vektörel kaydet — .png pikseldir, büyütünce bozulur.

İpucu

Matplotlib'i ezberlemeye çalışma; mantığını kavra: her şey bir Figure (tuval) ve bir veya çok Axes (grafik alanı) üzerinedir. Bir grafik türü lazım olduğunda “matplotlib [grafik adı] example” diye ara — galeri (matplotlib.org/stable/gallery) neredeyse her ihtiyaca hazır, kopyalanabilir örnek sunar.

16. Pandas ile Hızlı Çizim

Pandas, keşif sırasında hızlı grafikler için Matplotlib'i doğrudan sarar. Bir DataFrame veya Series üzerinde .plot() çağırman yeterlidir.

```

1 df['salary'].plot(kind='hist', bins=20, title='Maas Dagilimi')
2 df['salary'].plot(kind='box')
3 df.plot(x='age', y='salary', kind='scatter')
4 df.groupby('city')['salary'].mean().plot(kind='bar')
5 df['sales'].plot(kind='line') # zaman serisi
6
7 # kind secenekleri: 'line', 'bar', 'barh', 'hist', 'box', 'kde', 'area', 'scatter', 'pie'

```

17. Seaborn: İstatistiksel Görselleştirme

Seaborn, Matplotlib üzerine kurulu, istatistiksel grafikler için tasarlanmış bir kütüphanedir. Daha az kodla daha güzel ve bilgilendirici grafikler üretir; DataFrame'lerle doğrudan çalışır ve kategorik değişkenleri anlar.

17.1 Dağılım Grafikleri

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 sns.set_theme(style='whitegrid') # sik varsayilan tema
5
6 # Histogram + yogunluk egrisi
7 sns.histplot(data=df, x='salary', kde=True, bins=20)
8

```

```

9 # Yogunluk (KDE) grafigi
10 sns.kdeplot(data=df, x='salary', hue='city', fill=True)
11
12 # Kutu ve keman grafikleri (grup karsilastirma)
13 sns.boxplot(data=df, x='city', y='salary')
14 sns.violinplot(data=df, x='city', y='salary') # kutu + yogunluk
15
16 plt.show()

```

17.2 İlişki Grafikleri

İki değişken arasındaki ilişkiyi görselleştirmek, korelasyon ve modelleme için kritiktir.

```

1 # Dagilim + regresyon dogrusu
2 sns.scatterplot(data=df, x='age', y='salary', hue='city', size='salary')
3 sns.regplot(data=df, x='age', y='salary') # dogrusal uyum + guven araligi
4
5 # Pairplot: TUM sayisal sutun ciftleri (kesif icin harika)
6 sns.pairplot(df, hue='city')
7
8 # Isi haritasi (heatmap) -- korelasyon matrisi icin ideal
9 correlation = df.corr(numeric_only=True)
10 sns.heatmap(correlation, annot=True, cmap='coolwarm', center=0)

```

17.3 Kategorik Grafikler

```

1 sns.barplot(data=df, x='city', y='salary') # ortalama + guven araligi
2 sns.countplot(data=df, x='city') # kategori sayimi
3 sns.catplot(data=df, x='city', y='salary', # panel katmanli
4             kind='box', col='age_group')

```

İpucu

Keşifsel analiz (EDA) için Seaborn'un pairplot ve heatmap fonksiyonları paha biçilmezdir: pairplot tüm değişken çiftlerinin ilişkisini tek bakışta gösterir; korelasyon heatmap'i hangi değişkenlerin birbiriyle ilişkili olduğunu (ve modelde çoklu bağlantı riskini) anında ortaya koyar. Yeni bir veri setinde ilk çalıştıracağın şeyler bunlar olmalı.

KISIM 4

İstatistik ve Olasılık

18. Betimsel İstatistik

İstatistik, veri biliminin teorik temelidir. Betimsel istatistik, bir veri setini birkaç sayıyla *özetler*: merkezi eğilim (veri nerede toplanıyor?) ve yayılım (ne kadar dağınık?). Bu özetler, veriyi anlamının ilk adımıdır.

18.1 Merkezi Eğilim: Ortalama, Medyan, Mod

Ölçü	Tanım	Özellik
Ortalama (mean)	Tüm değerlerin toplamı / sayısı	Aykırı değerlerden <i>etkilenir</i>
Medyan (median)	Sıralı verinin ortadaki değeri	Aykırı değere <i>dayanıklı</i>
Mod (mode)	En sık geçen değer	Kategorik veride kullanışlı

```

1 import numpy as np
2 data = np.array([10, 12, 12, 13, 15, 18, 100]) # 100 bir aykırı değer
3
4 np.mean(data) # 25.7 -> aykırı değer yukarı çekti!
5 np.median(data) # 13.0 -> daha temsili
6 from scipy import stats
7 stats.mode(data, keepdims=False).mode # 12
8
9 # Pandas ile
10 df['salary'].mean(); df['salary'].median(); df['salary'].mode()

```

Ortalama mı medyan mı?

Veri *simetrikse* ortalama ve medyan birbirine yakındır. Ama veri *çarpıksa* (skewed — örneğin gelir, ev fiyatları) veya aykırı değerler varsa, medyan daha temsildir. “Ortalama maaş” bir CEO yüzünden yanıltıcı olabilir; “medyan maaş” gerçeği daha iyi yansıtır.

18.2 Yayılım: Varyans, Standart Sapma, Çeyrekler

Yayılım, verinin ne kadar “dağınık” olduğunu ölçer. İki veri seti aynı ortalamaya sahip olup çok farklı yayılıma sahip olabilir.

```

1 np.var(data)          # varyans: ortalamadan sapmaların karesinin ortalaması
2 np.std(data)          # standart sapma: varyansın karekökü (aynı birimde)
3 np.ptp(data)          # aralık (range): max - min
4
5 # Ceyrekler ve IQR (ceyrekler arası aralık)
6 q1 = np.percentile(data, 25)      # 1. ceyrek
7 q3 = np.percentile(data, 75)      # 3. ceyrek
8 iqr = q3 - q1                     # orta %50'nin yayılımı
9
10 # Aykırı değer tespiti (IQR yöntemi)
11 lower_bound = q1 - 1.5 * iqr
12 upper_bound = q3 + 1.5 * iqr
13 outliers = data[(data < lower_bound) | (data > upper_bound)]

```

Dikkat

NumPy varsayılan olarak *anakütle* (population) varyansını hesaplar (N 'e böler). *Örneklem* (sample) varyansı için ($N-1$ 'e bölme, yansız tahmin) `np.var(data, ddof=1)` kullan. Pandas ise varsayılan olarak örneklem varyansını ($ddof=1$) kullanır — bu fark küçük örneklemelerde önemlidir.

19. Olasılık Dağılımları

Olasılık dağılımı, bir rastgele değişkenin hangi değerleri hangi olasılıkla aldığını tarif eder. Veri bilimi modellerinin çoğu, verinin belirli bir dağılımdan geldiğini varsayar. En önemlisi *normal dağılımdır*.

19.1 Normal (Gauss) Dağılımı

Normal dağılım (çan eğrisi), doğada ve veride en sık karşılaşılan dağılımdır: boy, ölçüm hataları, birçok doğal olgu buna yakındır. İki parametresi vardır: ortalama μ (merkez) ve standart sapma σ (genişlik).

```

1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4
5 # Normal dagilimdan orneklem uret
6 samples = np.random.normal(loc=170, scale=10, size=1000) # mu=170, sigma=10
7
8 # Olasilik yogunluk fonksiyonu (PDF)
9 x = np.linspace(140, 200, 100)
10 pdf = stats.norm.pdf(x, loc=170, scale=10)
11 plt.plot(x, pdf)
12 plt.hist(samples, bins=30, density=True, alpha=0.5)
13
14 # Kumulatif olasilik: boyu 180'den kucuk olma olasiligi
15 stats.norm.cdf(180, loc=170, scale=10) # ~0.84

```

Bilgi

68-95-99.7 kuralı: Normal dağılımda verinin %68'i ortalamadan $\pm 1\sigma$, %95'i $\pm 2\sigma$, %99.7'si $\pm 3\sigma$ içindedir. Bu yüzden “ 3σ dışı” bir gözlem çok nadirdir (%0.3) ve genelde aykırı değer sayılır.

19.2 Diğer Önemli Dağılımlar

Dağılım	Ne modeller?
Normal	Sürekli, simetrik olgular (boy, hata).
Binom	N denemede başarı sayısı (yazı-tura).
Poisson	Sabit aralıkta olay sayısı (saatte gelen çağrı).
Üstel (exponential)	Olaylar arası bekleme süresi.
Düzgün (uniform)	Tüm sonuçların eşit olasılıklı olduğu durum.

19.3 Merkezi Limit Teoremi

Merkezi Limit Teoremi (MLT), istatistiğin en güçlü sonucudur: *hangi dağılımdan gelirse gelsin*, yeterince büyük örneklemelerin **ortalamaları** normal dağılıma yaklaşır. Bu, birçok istatistiksel testin neden çalıştığını açıklar.

```

1 # Kaynak dağılım: tamamen carpick (ustel) -- normal DEGİL
2 means = []
3 for _ in range(1000):
4     sample = np.random.exponential(scale=2, size=50) # 50'lik örnek
5     means.append(sample.mean()) # ortalamasını al
6
7 # means'in histogramı -> CAN EGRISINE benzer! (MLT)
8 plt.hist(means, bins=30)

```

20. Korelasyon ve Hipotez Testleri

20.1 Korelasyon: İlişki Ölçümü

Korelasyon, iki değişkenin birlikte nasıl değiştiğini -1 ile $+1$ arasında ölçer. $+1$ mükemmel pozitif, -1 mükemmel negatif, 0 ilişki yok demektir.

```

1 # Pearson korelasyonu (dogrusal iliski)
2 df['age'].corr(df['salary'])
3
4 # Tum sayisal sutunlari korelasyon matrisi
5 df.corr(numeric_only=True)
6
7 from scipy import stats
8 r, p = stats.pearsonr(df['age'], df['salary']) # katsayi + p-degeri
9 rho, p = stats.spearmanr(df['age'], df['salary']) # siralamaya dayali (dogrusal olmayan)

```

Dikkat

Korelasyon nedensellik değildir! İki değişken güçlü ilişkili olabilir ama biri diğerine *neden* olmuyor olabilir — ikisini birden etkileyen üçüncü bir faktör (confounder) olabilir. Klasik örnek: dondurma satışı ve boğulma vakaları ilişkilidir, ama sebep ikisini de artıran sıcak havadır.

20.2 Hipotez Testleri

Hipotez testi, bir iddianın (örneğin “iki grubun ortalaması farklı”) veriyle desteklenip desteklenmediğini istatistiksel olarak sorgular. Mantık: bir *sıfır hipotezi* (H_0 : fark yok) kurulur; veri bu hipotez altında çok olası değilse (p -değeri küçükse) H_0 reddedilir.

```
1 from scipy import stats
2
3 # t-testi: iki grubun ortalamasi farkli mi?
4 group_a = df[df['city'] == 'Ankara']['salary']
5 group_b = df[df['city'] == 'Izmir']['salary']
6 t_stat, p = stats.ttest_ind(group_a, group_b)
7
8 if p < 0.05:
9     print("Fark istatistiksel olarak anlamlı (H0 reddedildi)")
10 else:
11     print("Anlamlı fark yok")
12
13 # Ki-kare testi: iki kategorik degisken bagimsiz mi?
14 table = pd.crosstab(df['city'], df['age_group'])
15 chi2, p, dof, expected = stats.chi2_contingency(table)
16
17 # ANOVA: ikiden fazla grubun ortalamasi farkli mi?
18 f_stat, p = stats.f_oneway(group1, group2, group3)
```

p-değeri nedir?

p -değeri, " H_0 doğruysa, gözlemlediğimiz kadar (veya daha) uç bir sonucu görme olasılığıdır". Küçük p (< 0.05 yaygın eşik), sonucun şans eseri olma ihtimalinin düşük olduğunu, yani H_0 'a karşı kanıt olduğunu gösterir. Ama $p < 0.05$ "kesin doğru" demek değildir; sadece bir kanıt eşigidir ve dikkatli yorumlanmalıdır.

KISIM 5

Makine Öğrenmesi (scikit-learn)

21. Makine Öğrenmesine Giriş

Makine öğrenmesi (ML), açıkça programlamak yerine *veriden örüntü öğrenen* algoritmalar bütünüdür. Bir model, geçmiş veriye “bakarak” bir görevde (tahmin, sınıflandırma) ustalaşır. scikit-learn, klasik ML için Python’ın standart kütüphanesidir ve tutarlı bir arayüz sunar.

21.1 ML Türleri

Tür	Tanım	Örnek
Denetimli (regresyon)	Etiketli veriden sürekli değer tahmini	Ev fiyatı
Denetimli (sınıflandırma)	Etiketli veriden kategori tahmini	Spam mı değil mi
Denetimsiz (kümeleme)	Etiketsiz veride grup bulma	Müşteri segmentleri
Boyut indirgeme	Değişken sayısını azaltma	PCA ile sıkıştırma

21.2 scikit-learn’in Tutarlı Arayüzü

scikit-learn’ün gücü tutarlılığındadır: her model aynı üç metodu paylaşır. Bir kez öğrenince tüm modelleri kullanabilirsin.

Metot	Ne yapar?
<code>.fit(X, y)</code>	Modeli eğitim verisiyle eğitir (öğrenir).
<code>.predict(X)</code>	Yeni veri için tahmin üretir.
<code>.transform(X)</code>	Veriyi dönüştürür (ölçekleme, kodlama vb.).
<code>.score(X, y)</code>	Modelin başarısını ölçer.

21.3 Eğitim/Test Ayrımı: Altın Kural

Bir modeli, eğitildiği veriyle değerlendirmek yanıltıcıdır — model o veriyi “ezberlemiş” olabilir. Bu yüzden veriyi **eğitim** ve **test** olarak ayırırız: model eğitimde öğrenir, hiç görmediği test verisinde ölçülür.


```

1 from sklearn.model_selection import train_test_split
2
3 X = df[['age', 'experience', 'education']] # ozellikler (girdi)
4 y = df['salary']                         # hedef (cikti)
5
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y,
8     test_size=0.2,          # %20 test, %80 egitim
9     random_state=42)        # tekrarlanabilirlik icin

```

Dikkat

Veri sızıntısından (data leakage) kaçın: Ölçekleme, eksik doldurma gibi ön işlemleri *sadece eğitim verisine* fit et, sonra hem eğitim hem teste transform uygula. Tüm veriye birden fit edersen, test bilgisi eğitime “sızır” ve başarıyı gerçekçi olmayan biçimde yüksek gösterir. Pipeline’lar (ileride) bunu otomatik önler.

22. Regresyon

Regresyon, sürekli bir sayısal değeri tahmin eder (fiyat, sıcaklık, maaş). En temel modeli *doğrusal regresyondur*: çıktının, girdilerin ağırlıklı toplamı olduğunu varsayar: $y = w_0 + w_1x_1 + \dots + w_nx_n$.

22.1 Doğrusal Regresyon

```

1 from sklearn.linear_model import LinearRegression
2 from sklearn.metrics import mean_squared_error, r2_score
3 import numpy as np
4
5 model = LinearRegression()
6 model.fit(X_train, y_train)          # katsayilari ogren
7
8 y_pred = model.predict(X_test)       # test verisinde tahmin et
9
10 # Ogrenilen parametreler
11 model.coef_          # her ozelligin agirligi (w1..wn)
12 model.intercept_     # sabit terim (w0)
13
14 # Degerlendirme
15 rmse = np.sqrt(mean_squared_error(y_test, y_pred)) # ort. hata (birimde)
16 r2 = r2_score(y_test, y_pred)                    # aciklanan varyans oranı (0-1)
17 print(f"RMSE: {rmse:.2f}, R^2: {r2:.3f}")

```

22.2 Regresyon Metrikleri

Metrik	Anlamı
MAE	Ortalama mutlak hata; yorumlaması kolay, aykırıya dayanıklı.
MSE / RMSE	Kare hata; büyük hataları daha çok cezalandırır.
R^2	Modelin açıkladığı varyans oranı; 1’e yakın = iyi, 0 = ortalama kadar.

22.3 Aşırı Öğrenme ve Düzenleştirme (Regularization)

Model eğitim verisini *ezberlerse* (aşırı öğrenme — overfitting), yeni veride kötü çalışır. Ridge ve Lasso, katsayıları küçük tutarak (düzenleştirme) bunu önler. Lasso ayrıca bazı katsayıları sıfırlayıp otomatik özellik seçimi yapar.

```
1 from sklearn.linear_model import Ridge, Lasso
2
3 ridge = Ridge(alpha=1.0)      # L2 duzenlilestirme (katsayilari kucultur)
4 lasso = Lasso(alpha=0.1)     # L1 (bazi katsayilari SIFIRLAR)
5 ridge.fit(X_train, y_train)
6
7 # Dogrusal olmayan iliskiler icin agac tabanli:
8 from sklearn.ensemble import RandomForestRegressor
9 rf = RandomForestRegressor(n_estimators=100, random_state=42)
10 rf.fit(X_train, y_train)
11 rf.feature_importances_      # hangi ozellik ne kadar onemli
```

Bilgi

Aşırı vs eksik öğrenme: Eğitimde iyi, testte kötü = aşırı öğrenme (model çok karmaşık). Hem eğitimde hem testte kötü = eksik öğrenme (model çok basit). İkisinin dengesi “bias-variance ödünleşimi”dir. Eğitim ve test skorlarını *birlikte* izlemek, hangisinde olduğunu gösterir.

23. Sınıflandırma

Sınıflandırma, bir örneğin hangi *kategoriye* ait olduğunu tahmin eder: spam/değil, hasta/sağlıklı, hangi rakam. Çıktı süreklinin aksine ayrık sınıflardır.

23.1 Lojistik Regresyon ve Diğer Modeller

Adına rağmen lojistik regresyon bir *sınıflandırma* algoritmasıdır: bir örneğin bir sınıfa ait olma *olasılığını* tahmin eder.

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.tree import DecisionTreeClassifier
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.svm import SVC
5 from sklearn.neighbors import KNeighborsClassifier
6
7 model = LogisticRegression(max_iter=1000)
8 model.fit(X_train, y_train)
9
10 y_pred = model.predict(X_test)      # sinif tahmini
11 probabilities = model.predict_proba(X_test) # her sinifin olasiligi
12
13 # Diger siniflandiricilar -- ayni arayuz!
14 tree = DecisionTreeClassifier(max_depth=5)
15 forest = RandomForestClassifier(n_estimators=100)
16 knn = KNeighborsClassifier(n_neighbors=5)
```

23.2 Sınıflandırma Metrikleri ve Karmaşıklık Matrisi

Doğruluk (accuracy) tek başına yanıltıcı olabilir — özellikle *dengesiz* veride (örneğin %99 sağlıklı). Bu yüzden precision, recall ve F1 kullanılır.

```

1 from sklearn.metrics import (accuracy_score, precision_score, recall_score,
2                               f1_score, confusion_matrix, classification_report)
3
4 accuracy_score(y_test, y_pred)      # dogru tahmin orani
5 precision_score(y_test, y_pred)     # pozitif dedigimden kacisi gercekten pozitif
6 recall_score(y_test, y_pred)       # gercek pozitiflerin kacini yakaladim
7 f1_score(y_test, y_pred)           # precision ve recall'un harmonik ortalamasi
8
9 print(confusion_matrix(y_test, y_pred))  # TP, FP, TN, FN tablosu
10 print(classification_report(y_test, y_pred)) # hepsi bir arada

```

Metrik

Ne zaman önemli?

Precision (kesinlik) Yanlış alarm pahalıysa (spam olmayanı spam demek).

Recall (duyarlılık) Kaçırmak pahalıysa (hasta olanı sağlıklı demek).

F1 Precision ve recall dengesi gerektiğinde.

Precision mı recall mı?

Bu, hatanın maliyetine bağlıdır. Kansere teşhisinde *recall* kritiktir: bir hastayı kaçırmak, yanlış alarmdan çok daha kötüdür. Spam filtresinde *precision* önemlidir: önemli bir maili spam'e atmak kötüdür. Hangisini optimize edeceğine problem karar verir.

24. Denetimsiz Öğrenme: Kümeleme ve PCA

Denetimsiz öğrenmede *etiket yoktur*; amaç veride gizli yapı bulmaktır. En yaygın iki görev: kümeleme (benzer örnekleri gruplama) ve boyut indirgeme.

24.1 K-Means Kümeleme

K-Means, veriyi k kümeye ayırır: her örneği en yakın küme merkezine atar, merkezleri günceller, tekrarlar. Müşteri segmentasyonu, görüntü sıkıştırma kullanır.

```

1 from sklearn.cluster import KMeans
2
3 model = KMeans(n_clusters=3, random_state=42, n_init=10)
4 clusters = model.fit_predict(X)      # her orneğin küme etiketi (0,1,2)
5 model.cluster_centers_               # küme merkezleri
6
7 # Kac küme? Dirsek (elbow) yöntemi
8 inertia = []
9 for k in range(1, 10):
10     km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
11     inertia.append(km.inertia_)      # küme içi toplam uzaklık
12 # inertia grafiginde 'dirsek' noktası ideal k'yi verir

```

24.2 PCA: Boyut İndirgeme

Temel Bileşen Analizi (PCA), çok sayıda değişkeni, bilginin çoğunu koruyarak az sayıda “bileşene” indirir. Görselleştirme (yüksek boyutu 2B'ye indirme) ve gürültü azaltma için kullanılır.

```

1 from sklearn.decomposition import PCA
2 from sklearn.preprocessing import StandardScaler
3
4 # PCA olcege duyarli -> ONCE standartlastir
5 X_scaled = StandardScaler().fit_transform(X)
6
7 pca = PCA(n_components=2)           # 2 bilezene indir
8 X_pca = pca.fit_transform(X_scaled)
9 pca.explained_variance_ratio_      # her bilesenin acikladigi varyans
10
11 # 2B'ye indirip gorsellestir
12 plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters)

```

25. Model Değerlendirme ve Pipeline'lar

25.1 Çapraz Doğrulama (Cross-Validation)

Tek bir eğitim/test ayrımı şansa bağlı olabilir. Çapraz doğrulama, veriyi k parçaya böler; her seferinde birini test, gerisini eğitim yapıp k kez ölçer ve ortalar. Daha güvenilir bir başarı tahmini verir.

```

1 from sklearn.model_selection import cross_val_score
2
3 scores = cross_val_score(model, X, y, cv=5, scoring='accuracy') # 5-katli
4 print(f"Ortalama: {scores.mean():.3f} (+/- {scores.std():.3f})")

```

25.2 Hiperparametre Ayarı: GridSearch

Modellerin, veriden öğrenilmeyen ama elle ayarlanan *hiperparametreleri* vardır (ağaç derinliği, komşu sayısı...). GridSearch, bir parametre ızgarasını sistematik deneyip en iyisini çapraz doğrulamayla bulur.

```

1 from sklearn.model_selection import GridSearchCV
2
3 param_grid = {
4     'n_estimators': [50, 100, 200],
5     'max_depth':    [5, 10, None]
6 }
7 search = GridSearchCV(RandomForestClassifier(random_state=42),
8                       param_grid, cv=5, scoring='f1')
9 search.fit(X_train, y_train)
10
11 search.best_params_      # en iyi parametre kombinasyonu
12 search.best_score_      # en iyi capraz dogrulama skoru
13 best_model = search.best_estimator_

```

25.3 Pipeline: Ön İşleme + Model Tek Zincirde

Pipeline, ön işleme adımlarını ve modeli tek bir nesnede birleştirir. Bu, hem kodu temizler hem de veri sızıntısını otomatik önler (ön işleme her katlamada sadece eğitime fit edilir).

```

1 from sklearn.pipeline import Pipeline
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.impute import SimpleImputer

```

```
4
5 pipeline = Pipeline([
6     ('impute', SimpleImputer(strategy='median')), # eksikleri doldur
7     ('scale', StandardScaler()), # standartlastir
8     ('model', LogisticRegression(max_iter=1000)) # tahmin et
9 ])
10
11 pipeline.fit(X_train, y_train) # tum zincir egitilir
12 pipeline.predict(X_test) # tum zincir uygulanir
13 # GridSearchCV ile de dogrudan kullanilabilir
```

İpucu

Gerçek projelerde **her zaman Pipeline kullan**. Ön işleme adımlarını modelle birleştirmek; veri sızıntısını önler, kodu tekrardan kurtarır ve modeli tek parça olarak kaydedip (joblib.dump) üretime almayı kolaylaştırır. “Manuel” ön işleme, hataya en açık kısımdır.

KISIM 6

Uygulamalı Projeler: Gerçek Veri Setleri

Buraya kadar araçları tek tek öğrendik. Şimdi hepsini *gerçek, adı belli, indirilebilir* veri setleri üzerinde birleştiriyoruz. Aşağıdaki projelerin her biri kendi başına çalışır: veri seti bir tek satırla yüklenir (hepsi `seaborn` veya `scikit-learn` ile birlikte gelir ya da internetten çekilir), böylece kopyala-çalıştır-öğren yapabilirsin. Her projenin başında verinin kaynağı ve resmî linki verilmiştir.

Bilgi

Kullanılan veri setleri ve kaynakları:

- **Palmer Penguins** — allisonhorst.github.io/palmerpenguins
(seaborn: `sns.load_dataset('penguins')`)
- **Titanic** — kaggle.com/c/titanic
(seaborn: `sns.load_dataset('titanic')`)
- **Tips (Bahşiş)** — github.com/mwaskom/seaborn-data
(seaborn: `sns.load_dataset('tips')`)
- **California Housing** — [scikit-learn](https://scikit-learn.org/stable/datasets/real_world.html#california-housing) — California Housing
(`sklearn.datasets.fetch_california_housing`)

İpucu

`sns.load_dataset()` veriyi internetten çeker (bir kereliğine indirir). İnternet yoksa aynı CSV'leri github.com/mwaskom/seaborn-data adresinden indirip `pd.read_csv('penguins.csv')` ile açabilirsin.

26. Proje 1 — Palmer Penguins: Keşifsel Veri Analizi

Veri seti

Palmer Penguins: Antarktika'daki üç penguen türünün (Adelie, Chinstrap, Gentoo) gaga uzunluğu, kanat boyu, kütle gibi ölçümleri. Iris'e modern bir alternatif. Kaynak: allisonhorst.github.io/palmerpenguins.

Amaç: yeni bir veri setiyle ilk karşılaşmada izlenen *keşif* adımlarını (yükle → tanı → temizle → görselleştir → grupta) baştan sona uygulamak.

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import pandas as pd
4
5 # 1. YUKLE
6 df = sns.load_dataset('penguins')
7
8 # 2. ILK BAKIS
9 print(df.shape)           # (344, 7)
10 print(df.head())
11 print(df.info())          # sutun tipleri + eksik degerler
12 print(df.describe())      # sayisal ozet
13 print(df['species'].value_counts())
14
15 # 3. EKSİK VERİ ANALİZİ VE TEMİZLEME
16 print(df.isna().sum())    # her sutunda kac eksik?
17 df = df.dropna().reset_index(drop=True) # basit yol: eksikleri at
18 # (Alternatif: sayisallari medyan, kategorikleri mod ile doldurmak)
19
20 # 4. GRUPLA & TOPLULASTIR: tur bazinda ortalama olcumler
21 ozet = df.groupby('species')[['bill_length_mm', 'flipper_length_mm',
22                                'body_mass_g']].mean().round(1)
23 print(ozet)
24
25 # Tur + cinsiyet kiriliminda ortalama kutle
26 print(df.pivot_table(values='body_mass_g',
27                          index='species', columns='sex', aggfunc='mean'))
28
29 # 5. GORSELLESTIR: dagilimlar ve iliskiler
30 sns.set_theme(style='whitegrid')
31 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
32
33 sns.scatterplot(data=df, x='bill_length_mm', y='flipper_length_mm',
34                  hue='species', style='species', s=60, ax=axes[0])
35 axes[0].set_title('Gaga vs Kanat: turler ayrisir mi?')
36
37 sns.boxplot(data=df, x='species', y='body_mass_g', ax=axes[1])
38 axes[1].set_title('Ture gore vucut kutlesi')
39 fig.tight_layout(); plt.show()
40
41 # Tum sayisal ciftler + tur rengi (tek bakista kesif)
42 sns.pairplot(df, hue='species', height=1.8)
43 plt.show()

```

İpucu

Dağılım grafiğinde türlerin net kümeler oluşturduğunu göreceksin — bu, bir sınıflandırma modelinin türü ölçümlerden *kolayca* tahmin edebileceğinin işaretidir. Keşif aşaması, modellemeden önce “bu problem çözülebilir mi?” sorusunu yanıtlar.

27. Proje 2 — Titanic: Temizleme ve Sınıflandırma

Veri seti

Titanic: 1912 kazasındaki yolcuların yaş, cinsiyet, bilet sınıfı, ücret gibi bilgileri ve hayatta kalıp kalmadıkları. Makine öğrenmesinin “merhaba dünya”sı. Kaynak: kaggle.com/c/titanic.

Amaç: *gerçekçi biçimde dağınak* bir veriyi (eksik yaşlar, kategorik sütunlar) temizleyip özellik üretmek ve bir yolcunun hayatta kalıp kalmayacağını tahmin eden bir model kurmak.

```
1 import seaborn as sns
2 import pandas as pd
3
4 df = sns.load_dataset('titanic')
5 print(df[['survived', 'pclass', 'sex', 'age', 'fare', 'embarked']].head())
6 print(df.isna().sum())      # age ve deck çok eksik
7
8 # --- TEMİZLEME & OZELLİK MUHENDİSLİĞİ ---
9 d = df[['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked']].copy()
10
11 # Eksik yası, aynı pclass+sex grubunun MEDYANI ile doldur (akilli imputation)
12 d['age'] = d.groupby(['pclass', 'sex'])['age'].transform(
13     lambda s: s.fillna(s.median()))
14 d['embarked'] = d['embarked'].fillna(d['embarked'].mode()[0])
15
16 # Yeni özellikler: aile büyüklüğü ve yalnız mı?
17 d['family_size'] = d['sibsp'] + d['parch'] + 1
18 d['is_alone'] = (d['family_size'] == 1).astype(int)
19
20 # Kategorikleri sayısala çevir (one-hot)
21 d = pd.get_dummies(d, columns=['sex', 'embarked'], drop_first=True)
22 print(d.head())
```

```
1 # --- KESİF: hayatta kalma oranları ---
2 print(df.groupby('sex')['survived'].mean())      # kadınlar çok daha yüksek
3 print(df.groupby('pclass')['survived'].mean())  # 1. sınıf avantajlı
4
5 import matplotlib.pyplot as plt
6 sns.barplot(data=df, x='pclass', y='survived', hue='sex')
7 plt.title('Sınıf ve cinsiyete göre hayatta kalma oranı'); plt.show()
8
9 # --- MODELLEME: Lojistik Regresyon ---
10 from sklearn.model_selection import train_test_split
11 from sklearn.linear_model import LogisticRegression
12 from sklearn.preprocessing import StandardScaler
13 from sklearn.pipeline import Pipeline
14 from sklearn.metrics import classification_report, confusion_matrix
15
16 X = d.drop(columns='survived')
17 y = d['survived']
18 X_tr, X_te, y_tr, y_te = train_test_split(
19     X, y, test_size=0.2, random_state=42, stratify=y)
20
21 model = Pipeline([('scale', StandardScaler()),
22                  ('clf', LogisticRegression(max_iter=1000))])
23 model.fit(X_tr, y_tr)
24
25 print(f"Test doğruluğu: {model.score(X_te, y_te):.3f}")
26 print(classification_report(y_te, model.predict(X_te)))
27 print(confusion_matrix(y_te, model.predict(X_te)))
```



```

28
29 # Hangi özellik ne kadar etkili? (katsayılar)
30 katsayilar = pd.Series(model.named_steps['clf'].coef_[0], index=X.columns)
31 print(katsayilar.sort_values())

```

Dikkat

Eksik age değerlerini *tüm* sütunun ortalamasıyla doldurmak kolaydır ama kabadır. Yukarıdaki gibi `groupby(['pclass', 'sex'])` medyanıyla doldurmak, birinci sınıf bir kadının yaşını üçüncü sınıf bir erkeğinkinden farklı tahmin ederek çok daha gerçekçi olur. İyi *imputation* model başarısını doğrudan artırır.

28. Proje 3 — Tips: İstatistik ve Görselleştirme

Veri seti

Tips (Bahşiş): Bir restoranda 244 hesabın tutarı, bahşiş, gün, öğün, masa büyüklüğü. Görselleştirme ve ilişki analizi için ideal. Kaynak: github.com/mwaskom/seaborn-data.

Amaç: sürekli iki değişken (hesap tutarı ve bahşiş) arasındaki ilişkiyi görselleştirmek, korelasyonu ölçmek ve “bahşiş yüzdesi neye bağlı?” sorusunu gruplama ile yanıtlamak.

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 tips = sns.load_dataset('tips')
5 tips['tip_pct'] = tips['tip'] / tips['total_bill'] * 100 # bahsis yuzdesi
6
7 # --- KORELASYON ---
8 print(tips[['total_bill', 'tip', 'size', 'tip_pct']].corr().round(2))
9 # total_bill ile tip guclu pozitif; ama tip_pct ile zayif!
10
11 # --- GORSELLESTIR ---
12 sns.set_theme(style='whitegrid')
13 fig, ax = plt.subplots(1, 3, figsize=(15, 4))
14
15 # 1) Hesap vs bahsis + regresyon dogrusu, cinsiyete gore
16 sns.regplot(data=tips, x='total_bill', y='tip', ax=ax[0])
17 ax[0].set_title('Hesap arttikca bahsis artar')
18
19 # 2) Gun ve ogune gore ortalama bahsis yuzdesi
20 sns.barplot(data=tips, x='day', y='tip_pct', hue='time', ax=ax[1])
21 ax[1].set_title('Gun/ogune gore bahsis %')
22
23 # 3) Sigara icen/icmeyen dagilimi
24 sns.violinplot(data=tips, x='smoker', y='tip_pct', ax=ax[2])
25 ax[2].set_title('Sigara ve bahsis %')
26 fig.tight_layout(); plt.show()

```

```

1 # --- GRUPLAMA ile ICGORU ---
2 print(tips.groupby('day')['tip_pct'].agg(['mean', 'count']).round(2))
3 print(tips.groupby(['time', 'sex'])['tip_pct'].mean().round(2))
4
5 # Basit hipotez: ogle mi akşam mi daha comert?
6 from scipy import stats

```

```

7 ogle = tips.loc[tips['time']=='Lunch', 'tip_pct']
8 aksam = tips.loc[tips['time']=='Dinner', 'tip_pct']
9 t, p = stats.ttest_ind(ogle, aksam, equal_var=False)
10 print(f"t={t:.2f}, p={p:.3f} -> p<0.05 ise fark anlamlı")

```

İpucu

Dikkat çekici bulgu: `total_bill` ile `tip` güçlü ilişkilidir (büyük hesap → büyük bahşiş), ama bahşiş *yüzdesi* hesap tutarından neredeyse bağımsızdır. Ham korelasyon ile “oran” bazlı analiz farklı hikâyeler anlatır — doğru soruyu sormak, doğru grafiği seçmek kadar önemlidir.

29. Proje 4 — California Housing: Regresyon

Veri seti

California Housing: 1990 nüfus sayımından 20.640 bölgenin medyan gelir, ev yaşı, oda sayısı, konum bilgisi ve *medyan ev fiyatı*. Sürekli bir hedefi (fiyat) tahmin etmek için klasik regresyon verisi. Kaynak: [scikit-learn — California Housing](#).

Amaç: sürekli bir değeri (ev fiyatı) tahmin eden bir regresyon modeli kurmak, doğru metriklerle (MAE, RMSE, R^2) değerlendirmek ve hangi özelliğin fiyatı en çok belirlediğini bulmak.

```

1 from sklearn.datasets import fetch_california_housing
2 import pandas as pd
3
4 data = fetch_california_housing(as_frame=True)
5 df = data.frame
6 print(df.head())
7 print(df.describe())
8 # Hedef: MedHouseVal (100.000 dolar biriminde medyan ev degeri)
9
10 # Hangi ozellik fiyatla en iliskili?
11 print(df.corr()['MedHouseVal'].sort_values(ascending=False))
12 # MedInc (medyan gelir) acik ara en guclu belirleyici

```

```

1 from sklearn.model_selection import train_test_split, cross_val_score
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.linear_model import LinearRegression
4 from sklearn.ensemble import RandomForestRegressor
5 from sklearn.pipeline import Pipeline
6 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
7 import numpy as np
8
9 X = df.drop(columns='MedHouseVal')
10 y = df['MedHouseVal']
11 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)
12
13 # İki modeli karşılaştır: doğrusal (basit) vs random forest (güçlü)
14 modeller = {
15     'Linear': Pipeline([('sc', StandardScaler()), ('m', LinearRegression())]),
16     'RandomForest': RandomForestRegressor(n_estimators=100, random_state=42,
17                                         n_jobs=-1),
18 }
19 for ad, model in modeller.items():
20     model.fit(X_tr, y_tr)

```

```

21 tahmin = model.predict(X_te)
22 mae = mean_absolute_error(y_te, tahmin)
23 rmse = np.sqrt(mean_squared_error(y_te, tahmin))
24 r2 = r2_score(y_te, tahmin)
25 print(f"{ad:14s} MAE={mae:.3f} RMSE={rmse:.3f} R2={r2:.3f}")
26
27 # En iyi modelin özellik önemleri
28 rf = modeller['RandomForest']
29 onem = pd.Series(rf.feature_importances_, index=X.columns)
30 print(onem.sort_values(ascending=False))

```

İpucu

İki modeli karşılaştırdığında Random Forest'ın doğrusal modeli belirgin biçimde geçtiğini göreceksin — çünkü fiyat, özelliklerle doğrusal olmayan ve etkileşimli bir ilişki içinde (örn. konumun etkisi gelire bağlı değişir). Regresyonda da “basit bir temel çizgi (Linear) kur, sonra güçlü bir modelle kıyasla” altın kuraldır.

30. Kapanış: Uçtan Uca Bir Proje

Öğrendiğimiz her şeyi birleştiren tipik bir veri bilimi projesinin iskeleti şöyledir. Gerçek beceri, bu adımları veriye göre uyarlamaktır.

```

1 import pandas as pd
2 from sklearn.model_selection import train_test_split, cross_val_score
3 from sklearn.pipeline import Pipeline
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.ensemble import RandomForestClassifier
6 from sklearn.metrics import classification_report
7
8 # 1. VERİYİ YUKLE
9 df = pd.read_csv('veri.csv')
10
11 # 2. KESİF & TEMİZLE
12 df = df.drop_duplicates()
13 df['age'] = df['age'].fillna(df['age'].median())
14 print(df.describe())
15
16 # 3. ÖZELLİK & HEDEF AYIR
17 X = df.drop(columns=['target'])
18 y = df['target']
19
20 # 4. EGİTİM/TEST AYIR
21 X_train, X_test, y_train, y_test = train_test_split(
22     X, y, test_size=0.2, random_state=42, stratify=y)
23
24 # 5. PIPELINE KUR & EGİT
25 pipeline = Pipeline([
26     ('scale', StandardScaler()),
27     ('model', RandomForestClassifier(n_estimators=100, random_state=42))
28 ])
29 pipeline.fit(X_train, y_train)
30
31 # 6. DEĞERLENDİR
32 scores = cross_val_score(pipeline, X_train, y_train, cv=5)

```

```

33 print(f"CV doğruluk: {scores.mean():.3f}")
34 print(classification_report(y_test, pipeline.predict(X_test)))
35
36 # 7. KAYDET
37 import joblib
38 joblib.dump(pipeline, 'model.pkl')

```

30.1 Özet: Doğru Aracı Seçmek

Görev	Araç / Yöntem
Sayısal hesaplama, diziler	NumPy
Tablo verisi, temizleme	Pandas
Hızlı grafik	Pandas .plot(), Matplotlib
İstatistiksel grafik	Seaborn
Sürekli değer tahmini	Regresyon (Linear, RandomForest)
Kategori tahmini	Sınıflandırma (Logistic, RF, SVM)
Grup bulma (etiketsiz)	K-Means, boyut indirgeme (PCA)
Güvenilir değerlendirme	Çapraz doğrulama, Pipeline

İpucu

Veri biliminde başarının anahtarı fancy modeller değil, *temiz veri ve doğru değerlendirmedir*. Zamanının çoğu adım 2-3'te (temizleme, keşif) geçecek — ve bu normaldir. Basit bir modelle başla, temel bir başarı çizgisi (baseline) kur, sonra iyileştir. Bol gerçek veri setiyle pratik yapmak, bu sezgiyi geliştiren tek yoldur.